

《计算机系统基础》

第3章 程序的机器级表示 II：控制

授课老师:

何璐璐

luluhe@whu.edu.cn

上节回顾 -- 程序的机器级表示 I：基础

- 英特尔处理器和架构的历史
 - 汇编基础：寄存器、操作数、传送
 - 算术和逻辑运算
 - C、汇编、机器代码
-
- 控制：条件码
 - 条件分支

x86-64 整数寄存器

%rax	%eax
%rbx	%ebx
%rcx	%ecx
%rdx	%edx
%rsi	%esi
%rdi	%edi
%rsp	%esp
%rbp	%ebp

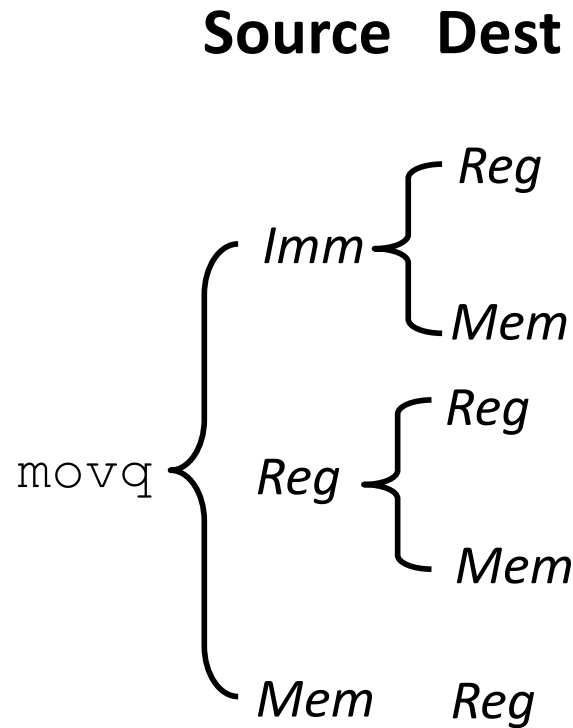
%r8	%r8d
%r9	%r9d
%r10	%r10d
%r11	%r11d
%r12	%r12d
%r13	%r13d
%r14	%r14d
%r15	%r15d

- Can reference low-order 4 bytes (also low-order 1 & 2 bytes)
- Not part of memory (or cache)

汇编特性：操作

- **在内存和寄存器之间传输数据**
 - 将数据从内存加载到寄存器
 - 将寄存器数据存入内存
- **对寄存器或存储器数据执行算术功能**
- **传送控制**
 - 无条件跳转到程序或跳出
 - 条件分支
 - 间接分支

movq 操作数组合



■ 通用形式

$D(Rb, Ri, S). \text{Mem}[\text{Reg}[Rb] + S * \text{Reg}[Ri] + D]$

- D: 恒定“位移”1、2 或 4 个字节
- Rb: 基址寄存器：16 个整数寄存器中的任何一个
- Ri: 变址寄存器：除了 **%rsp**
- S: 规模: 1, 2, 4, or 8

算术操作

■ 一元操作数指令：

incq	Dest
decq	Dest
negq	Dest
notq	Dest

■ 二元操作数指令

	Format	Computation
addq	Src, Dest	$\text{Dest} = \text{Dest} + \text{Src}$
subq	Src, Dest	$\text{Dest} = \text{Dest} - \text{Src}$
imulq	Src, Dest	$\text{Dest} = \text{Dest} * \text{Src}$
shlq	Src, Dest	$\text{Dest} = \text{Dest} \ll \text{Src}$
sarq	Src, Dest	$\text{Dest} = \text{Dest} \gg \text{Src}$
shrq	Src, Dest	$\text{Dest} = \text{Dest} \gg \text{Src}$
xorq	Src, Dest	$\text{Dest} = \text{Dest} \wedge \text{Src}$
andq	Src, Dest	$\text{Dest} = \text{Dest} \& \text{Src}$
orq	Src, Dest	$\text{Dest} = \text{Dest} \text{Src}$

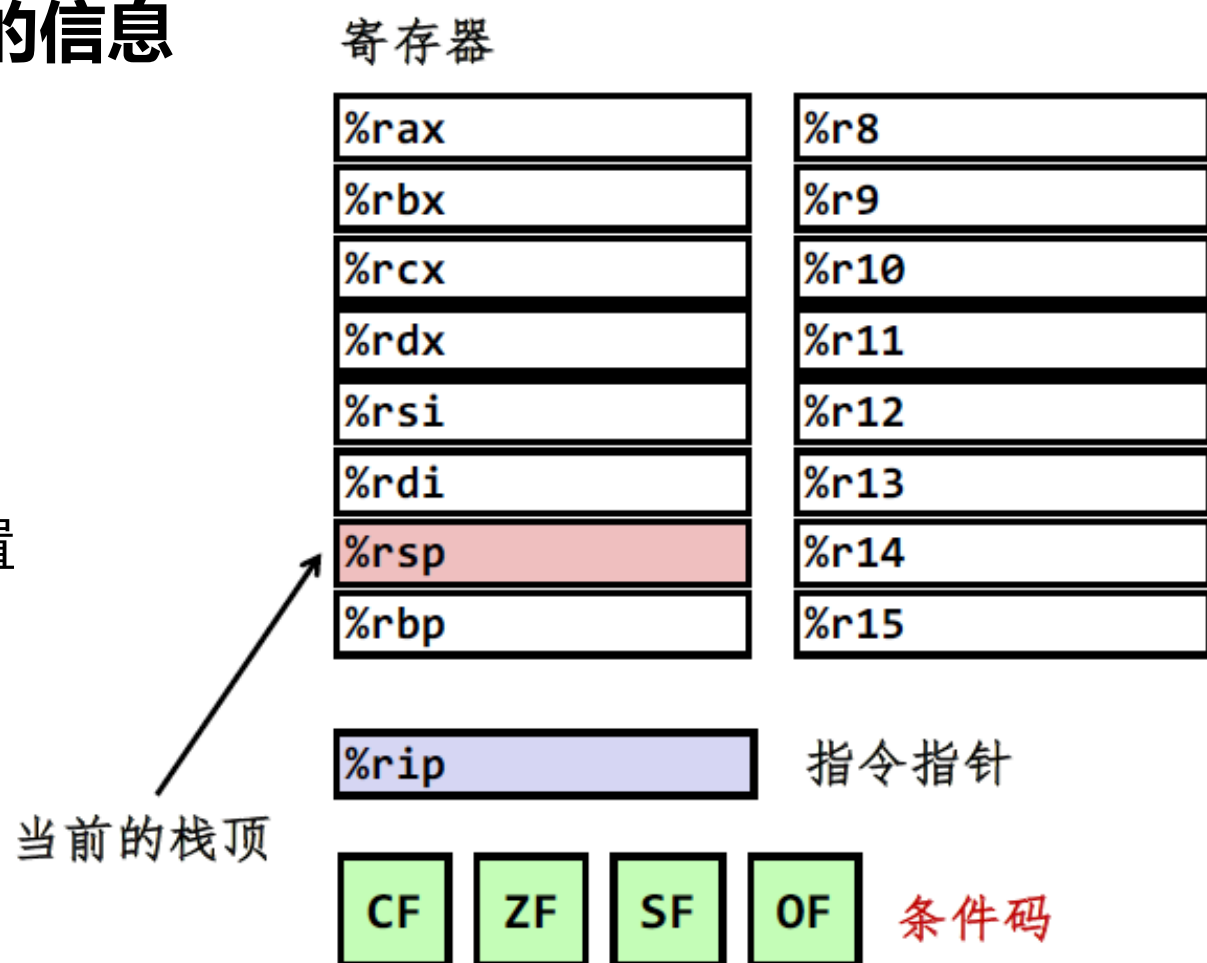
本节内容

- 控制：条件码
- 条件分支
- 循环
- Switch 语句

处理器状态（ x86-64 ， 部分 ）

■ 当前正在执行的程序的信息

- 临时数据
(%rax, ...)
- 运行时栈的位置
(%rsp)
- 当前代码控制点的位置
(%rip, ...)
- 最近一次测试的状态
(CF, ZF, SF, OF)



条件码（隐式设置）

■ 单比特位条件码寄存器

- | | | | |
|-------------|-----------|-----------|-----------|
| ■ CF | 进位标志（无符号） | SF | 符号标志（有符号） |
| ■ ZF | 零标志 | OF | 符号标志（有符号） |

■ 算术运算的隐式设置

示例: `addq Src, Dest` $\leftrightarrow$ `t = a+b`

设置**CF** 如果最高有效位产生进位/借位（无符号溢出）

设置**ZF** 如果 `t == 0`

设置**SF** 如果 `t < 0` (用于有符号数)

设置**OF** 如果补码溢出（有符号，正溢出或负溢出）

`(a>0 && b>0 && t<0) || (a<0 && b<0 && t>=0)`

■ **leaq** 指令不改变任何条件码

CF：进位标志

- 在x86 机器中，对减法操作，是将 CF 当作借位标志（borrow flag）来设置的
 - 当计算 $a-b$ 且 $a < b$ 时，必须进行借位
 - 如果 $a \geq b$ ，该位被清零
- 带借位减法指令(SBB)会计算 $a-b-C = a-(b+C)$
- 不带借位减法指令(SUB)计算时，会视借位被清零
- System/360, 6502, MSP430, COP8, ARM and PowerPC 处理器使用其他的约定方式
 - 它会直接使用等式 $-x = (\sim x)+1$ (i.e. 不会存储进位的反转值)，以 $a+(\sim b)+1$ 方式来计算 $a-b$ 。依据此加法设置The carry flag is set according to this addition.

条件码（显式设置：compare）

■ 通过 compare 指令来显式设置条件码

- `cmpq Src2, Src1`
- `cmpq b`, 就像执行 `a-b` 但不改变目的寄存器的值
- 设置 **CF** : 如果最高有效位发生进位/借位 (用于无符号比较)
- 设置 **ZF** : 如果 `a == b`
- 设置 **SF** : 如果 `(a-b) < 0` (用于有符号数)
- 设置 **OF** : 如果补码 (有符号) 溢出
`(a>0 && b<0 && (a-b)<0) || (a<0 && b>0 && (a-b)>0)`

条件码（显式设置：test）

■ 通过 test 指令进行显式设置

- `testq Src2, Src1`
 - `testq b, a` 类似于执行 `a&b` 但不改变目的寄存器的值
- 根据 Src1 和 Src2 的值设置条件码
- 当一个操作数为掩码时会很有用
- 设置 ZF：当 `a&b == 0`
- 设置 SF：当 `a&b < 0`

常用：

```
testq    %rax, %rax
```

读取条件码（显式读取： **set**）

■ 通过 Set 指令来显示读取条件码

- **setX** Dest：根据条件码不同组合的结果设置目的寄存器的最低字节为0或1
- **不改变**操作数 Dest 的剩余 7 个字节

SetX	条件码	描述
sete	ZF	相等/零
setne	~ZF	不等/非零
sets	SF	负数
setns	~SF	非负数
setg	$\sim (SF \wedge OF) \ \& \sim ZF$	大于（有符号>）
setge	$\sim (SF \wedge OF)$	大于等于（有符号>=）
setl	$SF \wedge OF$	小于（有符号<）
setle	$(SF \wedge OF) \mid ZF$	小于等于（有符号<=）
seta	$\sim CF \ \& \sim ZF$	超过（无符号>）
setb	CF	超过（无符号>）

x86-64 整型寄存器

■ 可以引用低位字节

%rax	%al
%rbx	%bl
%rcx	%cl
%rdx	%dl
%rsi	%sil
%rdi	%dil
%rsp	%spl
%rbp	%bpl

%r8	%r8b
%r9	%r9b
%r10	%r10b
%r11	%r11b
%r12	%r12b
%r13	%r13b
%r14	%r14b
%r15	%r15b

读取条件码（显式读取： **set**）

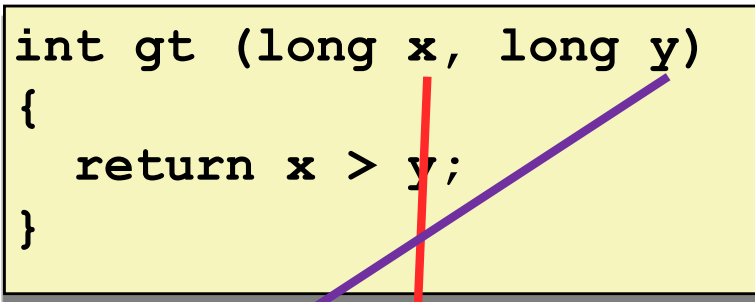
■ SetX 指令：

- 根据条件码组合设置单个的字节

■ 某个可寻址的字节寄存器

- 并不改变其它字节
- 比较典型的是用 `movzbl` 指令完成整个操作
 - 32位的指令也将高 32 位设置为 0

```
int gt (long x, long y)
{
    return x > y;
}
```



```
cmpq    %rsi, %rdi    # Compare x:y
setg     %al           # Set when >
movzbl   %al, %eax     # Zero rest of %rax
ret
```

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rax	Return value

本节内容

- 控制：条件码
- 条件分支
- 循环
- Switch 语句

跳转指令 (jump)

■ jX 指令

- 根据条件代码跳转到代码的不同部分
- 隐式读取条件代码

jX	跳转条件	描述
jmp	1	无条件跳跃
je	ZF	相等/零
jne	$\sim ZF$	不相等/非零
js	SF	负数
jns	$\sim SF$	非负数
jg	$\sim (SF \wedge OF) \ \& \ \sim ZF$	大于 (有符号 >)
jge	$\sim (SF \wedge OF)$	大于或等于 (有符号 $\geq$)
jl	$SF \wedge OF$	小于 (有符号 <)
jle	$(SF \wedge OF) \ \ ZF$	小于或等于 (有符号 $\leq$)
ja	$\sim CF \ \& \ \sim ZF$	超过 (无符号 >)
jb	CF	低于 (无符号 <)

条件分支示例 (旧版风格)

■ 通常情况

```
linux> gcc -Og -S -fno-if-conversion control.c
```

```
long absdiff(long x, long y)
{
    long result;
    if (x > y)
        result = x-y;
    else
        result = y-x;
    return result;
}
```

```
absdiff:
    cmpq    %rsi, %rdi    # x:y
    jle     .L4
    movq    %rdi, %rax
    subq    %rsi, %rax
    ret

.L4:      # x <= y
    movq    %rsi, %rax
    subq    %rdi, %rax
    ret
```

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rax	Return value

Goto 语句表达

- C 允许goto 语句
- 跳转到标号指定的位置

```
long absdiff(long x, long y)
{
    long result;
    if (x > y)
        result = x-y;
    else
        result = y-x;
    return result;
}
```

```
long absdiff_j(long x, long y)
{
    long result;
    int ntest = (x <= y);
    if (ntest) goto Else;
    result = x-y;
    goto Done;
Else:
    result = y-x;
Done:
    return result;
}
```

一般条件表达式的翻译（使用分支语句）

C 代码

```
val = Test ? Then_Expr : Else_Expr;
```

```
val = x > y ? x - y : y - x;
```

Goto 版本

```
n_test = !Test;  
if (n_test) goto Else;  
val = Then_Expr;  
goto Done;  
Else:  
    val = Else_Expr;  
Done:  
    . . .
```

- 为 then & else 表达式创建单独的代码区域
- 执行适当的那部分代码

使用条件传送

■ 条件传送指令

- 类似于：
if (Test) Dest $\leftarrow$ Src
- 在 1995 年后的 x86 处理器中支持
- GCC 会尝试使用
 - 仅在已知安全的情况下才可以使用

■ 原因

- 指令通过流水线时，分支指令会中断
- 指令流
- 条件传送指令不要求控制转移

C 代码

```
val = Test  
    ? Then_Expr  
    : Else_Expr;
```

Goto 版本: 条件分支

```
n_test = !Test;  
if (n_test) goto Else;  
val = Then_Expr;  
goto Done;  
Else:  
    val = Else_Expr;  
Done:    . . .
```

Goto 版本: 条件传送

```
result = Then_Expr;  
eval = Else_Expr;  
nt = !Test;  
if (nt) result = eval;  
return result;
```

条件转移示例

```
long absdiff (long x, long y)
{
    long result;
    if (x > y)
        result = x-y;
    else
        result = y-x;
    return result;
}
```

```
absdiff:
    movq    %rdi, %rax    # x
    subq    %rsi, %rax    # result = x-y
    movq    %rsi, %rdx
    subq    %rdi, %rdx    # eval = y-x
    cmpq    %rsi, %rdi    # x:y
    cmovle  %rdx, %rax    # if <=, result = eval
    ret
```

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rax	Return value

When is
this bad?

条件转移指令

- 当传送条件满足时，将源操作数的值s传送到目的寄存器或内存

Instruction	Condition	Description
cmove	ZF	相等/零
cmovne	~ZF	不相等/非零
cmovs	SF	负数
cmovns	~SF	非负数
cmovg	~ (SF^OF) & ~ZF	大于（有符号>）
cmovge	~ (SF^OF)	大于或等于（有符号>=）
cmovl	SF^OF	小于（有符号
cmovle	(SF^OF) ZF	小于或等于（有符号<=）
cmova	~CF&~ZF	超过（无符号>）
cmovae	~CF	超过或相等（无符号>=）
cmovb	CF	低于（无符号<）
cmovbe	CF ZF	低于或相等（无符号<=）

不适合条件传送的情况

计算昂贵

```
val = Test(x) ? Hard1(x) : Hard2(x);
```

- 所有的值都被计算
- 只有当计算很简单的时候才有好处

糟糕的性能

计算有风险性

```
val = p ? *p : 0;
```

- 所有的值都被计算
- 可能产生意想不到的效果

不安全性

计算可能产生副作用

```
val = x > 0 ? x*=7 : x+=3;
```

- 所有的值都被计算
- 必须是无副作用的才能使用

不合法性

本节内容

- 控制：条件码
- 条件分支
- 循环
- Switch 语句

“Do-While” 循环示例

C 代码

```
long pcount_do
(unsigned long x) {
    long result = 0;
    do {
        result += x & 0x1;
        x >>= 1;
    } while (x);
    return result;
}
```

Goto 版本

```
long pcount_goto
(unsigned long x) {
    long result = 0;
    loop:
        result += x & 0x1;
        x >>= 1;
        if(x) goto loop;
    return result;
}
```

- 计算参数 x 中 1 的数量 (“popcount”)
- 使用条件分支语句继续循环或退出循环

“Do-While” 循环示例

```
long pcount_goto
(unsigned long x) {
    long result = 0;
loop:
    result += x & 0x1;
    x >>= 1;
    if(x) goto loop;
    return result;
}
```

Register	Use(s)
%rdi	Argument x
%rax	result

```
        movl    $0, %eax    # result = 0
.L2:                                # loop:
        movq    %rdi, %rdx
        andl    $1, %edx    # t = x & 0x1
        addq    %rdx, %rax  # result += t
        shrq    %rdi        # x >>= 1
        jne     .L2         # if(x) goto loop
        rep; ret
```

“Do-While” 语句的通用形式

C Code

```
do  
    Body  
while (Test) ;
```

Goto Version

```
loop:  
    Body  
    if (Test)  
        goto loop
```

■ **Body:** {
 Statement_1 ;
 Statement_2 ;
 ...
 Statement_n ;
}

一般“While”语句转化#1

- “跳转到中间”翻译
- 使用 -Og编译选项

While 版本

```
while (Test)  
    Body
```



Goto 版本

```
    goto test;  
loop:  
    Body  
test:  
    if (Test)  
        goto loop;  
done:
```

While 循环示例 #1

C 代码

```
long pcount_while
(unsigned long x) {
    long result = 0;
    while (x) {
        result += x & 0x1;
        x >>= 1;
    }
    return result;
}
```

跳转到中间

```
long pcount_goto_jtm
(unsigned long x) {
    long result = 0;
    goto test;
loop:
    result += x & 0x1;
    x >>= 1;
test:
    if(x) goto loop;
    return result;
}
```

- 与函数的 do-while 版本进行比较
- 第一个goto 从test 处开始循环

一般While语句转化 #2

While version

```
while (Test)  
    Body
```



Do-While 版本

```
if (!Test)  
    goto done;  
do  
    Body  
    while (Test);  
done:
```



Goto 版本

```
if (!Test)  
    goto done;  
loop:  
    Body  
    if (Test)  
        goto loop;  
done:
```

- “Do-while” 转化
- 使用 -O1 编译选项

While 循环示例 #2

C 代码

```
long pcount_while
(unsigned long x) {
    long result = 0;
    while (x) {
        result += x & 0x1;
        x >>= 1;
    }
    return result;
}
```

Do-While 版本

```
long pcount_goto_dw
(unsigned long x) {
    long result = 0;
    if (!x) goto done;
loop:
    result += x & 0x1;
    x >>= 1;
    if(x) goto loop;
done:
    return result;
}
```

- 最开始的条件判断控制能否进入循环
- 与该函数的 do-while 版本进行比较
 - 删除了跳转到中间这个过程。是好是坏呢？

翻译方法 #1 vs. 翻译方法 #2

跳转到中间

```
long pcount_goto_jtm
(unsigned long x) {
    long result = 0;
    goto test;
loop:
    result += x & 0x1;
    x >>= 1;
test:
    if(x) goto loop;

    return result;
}
```

翻译方法#2

```
long pcount_goto_dw
(unsigned long x) {
    long result = 0;
    if (!x) goto done;
loop:
    result += x & 0x1;
    x >>= 1;

    if(x) goto loop;
done:
    return result;
}
```

“For” 循环形式

一般形式

```
for (Init; Test; Update )  
    Body
```

```
#define WSIZE 8*sizeof(int)  
long pcount_for  
    (unsigned long x)  
{  
    size_t i;  
    long result = 0;  
    for (i = 0; i < WSIZE; i++)  
    {  
        unsigned bit =  
            (x >> i) & 0x1;  
        result += bit;  
    }  
    return result;  
}
```

初始

```
i = 0
```

测试

```
i < WSIZE
```

更新

```
i++
```

内容

```
{  
    unsigned bit =  
        (x >> i) & 0x1;  
    result += bit;  
}
```

“For” 循环 → While 循环

For 版本

```
for (Init; Test; Update )  
    Body
```



While 版本

```
Init ;  
while (Test) {  
    Body  
    Update ;  
}
```

For-While 转换

初始

```
i = 0
```

测试

```
i < WSIZE
```

更新

```
i++
```

内容

```
{  
    unsigned bit =  
        (x >> i) & 0x1;  
    result += bit;  
}
```

```
Init;  
while (Test) {  
    Body  
    Update;  
}
```

```
long pcount_for_while  
(unsigned long x)  
{  
    size_t i;  
    long result = 0;  
    i = 0;  
    while (i < WSIZE)  
    {  
        unsigned bit =  
            (x >> i) & 0x1;  
        result += bit;  
        i++;  
    }  
    return result;  
}
```

“For” 循环向 Do-While 循环的转换

C 代码

```
long pcount_for
(unsigned long x)
{
    size_t i;
    long result = 0;
    for (i = 0; i < WSIZE; i++)
    {
        unsigned bit =
            (x >> i) & 0x1;
        result += bit;
    }
    return result;
}
```

- 初试的判断可以被优化掉

Goto 版本

```
long pcount_for_goto_dw
(unsigned long x) {
    size_t i;
    long result = 0;
    i = 0;
    if (!(i < WSIZE)) Init
    goto done; !Test
loop:
{
    unsigned bit =
        (x >> i) & 0x1; Body
    result += bit;
}
i++; Update
if (i < WSIZE) Test
    goto loop;
done:
    return result;
}
```

本节内容

- 控制：条件码
- 条件分支
- 循环
- Switch 语句

```

long my_switch
(long x, long y, long z)
{
    long w = 1;
    switch(x) {
    case 1:
        w = y*z;
        break;
    case 2:
        w = y/z;
        /* Fall Through */
    case 3:
        w += z;
        break;
    case 5:
    case 6:
        w -= z;
        break;
    default:
        w = 2;
    }
    return w;
}

```

Switch 语句示例

- 多个case 标号
 - 这里：5 & 6
- 下落的情况
 - 这里：2
- 缺失的情况
 - 这里：4

跳转表结构

Switch 形式

```
switch(x) {  
  case val_0:  
    Block 0  
  case val_1:  
    Block 1  
    . . .  
  case val_n-1:  
    Block n-1  
}
```

翻译 (拓展 C)

```
goto *JTab[x];
```

跳转表

jtab:	Targ0
	Targ1
	Targ2
	•
	•
	•
	Targn-1

跳转目标

Targ0:

Code Block
0

Targ1:

Code Block
1

Targ2:

Code Block
2

•
•
•

Targn-1:

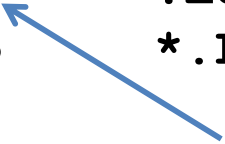
Code Block
n-1

Switch 语句示例

```
long my_switch(long x, long y, long z)
{
    long w = 1;
    switch(x) {
        . . .
    }
    return w;
}
```

Setup

```
my_switch:
    movq    %rdx, %rcx
    cmpq    $6, %rdi    # x:6
    ja      .L8
    jmp     *.L4(, %rdi, 8)
```



默认值的范围是多少？

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rdx	Argument z
%rax	Return value

注意 w 没有在这里初始化

Switch 语句示例

```
long my_switch(long x, long y, long z)
{
    long w = 1;
    switch(x) {
        . . .
    }
    return w;
}
```

跳转表

```
.section      .rodata
    .align 8
.L4:
    .quad     .L8      # x = 0
    .quad     .L3      # x = 1
    .quad     .L5      # x = 2
    .quad     .L9      # x = 3
    .quad     .L8      # x = 4
    .quad     .L7      # x = 5
    .quad     .L7      # x = 6
```

Setup

```
my_switch:
    movq      %rdx, %rcx
    cmpq      $6, %rdi    # x:6
    ja        .L8          # use default
    jmp       *.L4(,%rdi,8) # goto *Jtab[x]
```

间接跳转

汇编代码地址分配说明

■ 表的结构

- 每个目标地址需要 8 个字节
- .L4 处的基地址

■ 跳转

- **直接跳转** : `jmp .L8`
- 跳转目标由标签 .L8 表示
- **间接跳转** : `jmp *.L4(,%rdi,8)`
- 跳转表的起始地址: .L4
- 必须以8的倍数为单位来表示 (地址为 8 个字节)
- 从有效地址 `.L4 + x*8` 获取目标地址
 - 仅适用于 $0 \leq x \leq 6$

跳转表

```
.section      .rodata
    .align 8
.L4:
    .quad     .L8    # x = 0
    .quad     .L3    # x = 1
    .quad     .L5    # x = 2
    .quad     .L9    # x = 3
    .quad     .L8    # x = 4
    .quad     .L7    # x = 5
    .quad     .L7    # x = 6
```

跳转表

跳转表

```
.section .rodata
.align 8
.L4:
.quad .L8 # x = 0
.quad .L3 # x = 1
.quad .L5 # x = 2
.quad .L9 # x = 3
.quad .L8 # x = 4
.quad .L7 # x = 5
.quad .L7 # x = 6
```

```
switch(x) {
case 1:      // .L3
    w = y*z;
    break;
case 2:      // .L5
    w = y/z;
    /* Fall Through */
case 3:      // .L9
    w += z;
    break;
case 5:
case 6:      // .L7
    w -= z;
    break;
default:    // .L8
    w = 2;
}
```

The diagram illustrates the mapping between the jump table and the switch statement. Red arrows point from the jump table entries to the corresponding case labels in the switch statement. Green arrows point from the jump table entries to the corresponding code blocks in the switch statement.

代码块 (x == 1)

```
switch(x) {  
  case 1:      // .L3  
    w = y*z;  
    break;  
  . . .  
}
```

```
.L3:  
  movq    %rsi, %rax  # y  
  imulq   %rdx, %rax  # y*z  
  ret
```

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rdx	Argument z
%rax	Return value

如何解决下落的情况

```
long w = 1;  
.  
.  
.  
switch(x) {  
.  
.  
.  
case 2:  
    w = y/z;  
    /* Fall Through */  
case 3:  
    w += z;  
    break;  
.  
.  
.  
}
```

case 2:
 w = y/z;
 goto merge;

case 3:
 w = 1;
merge:
 w += z;

代码块 (x == 2, x == 3)

```
long w = 1;
. . .
switch(x) {
. . .
case 2:
    w = y/z;
    /* Fall Through */
case 3:
    w += z;
    break;
. . .
}
```

```
.L5:                                # Case 2
    movq    %rsi, %rax
    cqto                                # sign extend
                                # rax to rdx:rax
    idivq    %rcx                      # y/z
    jmp     .L6                        # goto merge
.L9:                                # Case 3
    movl     $1, %eax                 # w = 1
.L6:                                # merge:
    addq     %rcx, %rax               # w += z
    ret
```

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rcx	z
%rax	Return value

代码块 (x == 5, x == 6, 默认情况)

```
switch(x) {  
    . . .  
    case 5:  // .L7  
    case 6:  // .L7  
        w -= z;  
        break;  
    default: // .L8  
        w = 2;  
}
```

```
.L7:                                # Case 5,6  
    movl    $1, %eax                # w = 1  
    subq    %rdx, %rax              # w -= z  
    ret  
.L8:                                # Default:  
    movl    $2, %eax                # 2  
    ret
```

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rdx	Argument z
%rax	Return value

在二进制代码中查找跳转表

```
00000000004005e0 <switch_eg>:
4005e0:      48 89 d1                mov     %rdx,%rcx
4005e3:      48 83 ff 06           cmp     $0x6,%rdi
4005e7:      77 2b                   ja      400614 <switch_eg+0x34>
4005e9:      ff 24 fd f0 07 40 00  jmpq    *0x4007f0(,%rdi,8)
4005f0:      48 89 f0                mov     %rsi,%rax
4005f3:      48 0f af c2           imul    %rdx,%rax
4005f7:      c3                     retq
4005f8:      48 89 f0                mov     %rsi,%rax
4005fb:      48 99                   cqto
4005fd:      48 f7 f9               idiv    %rcx
400600:      eb 05                   jmp     400607 <switch_eg+0x27>
400602:      b8 01 00 00 00         mov     $0x1,%eax
400607:      48 01 c8               add     %rcx,%rax
40060a:      c3                     retq
40060b:      b8 01 00 00 00         mov     $0x1,%eax
400610:      48 29 d0               sub     %rdx,%rax
400613:      c3                     retq
400614:      b8 02 00 00 00         mov     $0x2,%eax
400619:      c3                     retq
```

在二进制代码中查找跳转表 (.续)

```
00000000004005e0 <switch_eg>:
```

```
. . .
```

```
4005e9:      ff 24 fd f0 07 40 00      jmpq    *0x4007f0(,%rdi,8)
```

```
. . .
```

```
% gdb switch
```

```
(gdb) x /8xg 0x4007f0
```

```
0x4007f0:      0x0000000000400614      0x00000000004005f0
```

```
0x400800:      0x00000000004005f8      0x0000000000400602
```

```
0x400810:      0x0000000000400614      0x000000000040060b
```

```
0x400820:      0x000000000040060b      0x2c646c25203d2078
```

```
(gdb)
```

在二进制代码中查找跳转表 (.续)

```
% gdb switch
(gdb) x /8xg 0x4007f0
0x4007f0:      0x000000000000400614      0x0000000000004005f0
0x400800:      0x0000000000004005f8      0x000000000000400602
0x400810:      0x000000000000400614      0x00000000000040060b
0x400820:      0x00000000000040060b      0x2c646c25203d2078
```

4005f0:	48 89 f0	mov	%rsi,%rax
4005f3:	48 0f af c2	imul	%rdx,%rax
4005f7:	c3	retq	
4005f8:	48 89 f0	mov	%rsi,%rax
4005fb:	48 99	cqto	
4005fd:	48 f7 f9	idiv	%rcx
400600:	eb 05	jmp	400607 <switch_eg+0x27>
400602:	b8 01 00 00 00	mov	\$0x1,%eax
400607:	48 01 c8	add	%rcx,%rax
40060a:	c3	retq	
40060b:	b8 01 00 00 00	mov	\$0x1,%eax
400610:	48 29 d0	sub	%rdx,%rax
400613:	c3	retq	
400614:	b8 02 00 00 00	mov	\$0x2,%eax
400619:	c3	retq	

总结

■ C 语言的控制

- if-then-else
- do-while
- while, for
- switch

■ 汇编语言的控制

- 条件跳转
- 有条件的移动
- 间接跳转（通过跳转表）
- 编译器生成代码序列以实现更复杂的控制

■ 标准技术

- 循环转换为 do-while 或 jump-to-middle（跳转到中间）形式
- 较大的 switch 语句使用跳转表
- 稀疏的 switch 语句可以使用决策树(if-elseif-elseif-else)